

# Per-Node Activation Function Evolution in Indirectly Encoded Substrates: Solvability, Limits, and Emergent Diversity

Romain Claret<sup>1</sup>, Michael O’Neill<sup>2</sup>, Paul Cotofrei<sup>1</sup>, and Kilian Stoffel<sup>1</sup>

<sup>1</sup>University of Neuchâtel, Switzerland

<sup>2</sup>University College Dublin, Ireland romain.claret@unine.ch

## Abstract

Biological neurons achieve computational diversity through specialized types: tonic, bursting, adapting, and fast-spiking cells coexist within the same circuit. Artificial neural networks, by contrast, apply a single activation function uniformly to all nodes, which limits what they can represent. We show that this uniformity creates hard limits for evolutionary search: across sparse evolved substrates, monotonic functions fail to solve parity beyond its smallest instance, XOR, while a single oscillatory unit suffices at all tested scales. The gap is one of search and sparsity, not representation: monotonic networks can represent parity with a modest number of hidden units, and gradient descent recovers that solution. We evolve, to our knowledge for the first time in indirect encoding, per-node activation function assignments from an 18-function palette across more than 4,500 experimental runs spanning Boolean logic, regression, and spatial classification.

Testing each of the 18 functions individually on Parity-4 reveals a three-tier solvability structure: oscillatory functions achieve 100%, intermediate functions 6.7–80%, and all 9 monotonic functions 0%. This divide is not universal. Recurrence collapses it, and gradient descent inverts it entirely, showing that the barrier is specific to evolutionary search in sparse substrates. What activation functions a network can use, beyond its topology and weights, determines what evolutionary search can solve. Indirect encoding discovers heterogeneous per-node activation assignments unlikely to be chosen by hand.

Data/Code available at: <https://github.com/RomainClaret/emr-hyperneat>

## Introduction

Every cortical circuit contains a mix of neuron types with distinct firing properties (Izhikevich, 2003; Markram et al., 2004), yet most artificial neural networks force every node to use the same activation function (LeCun et al., 2015). This uniformity is consequential. It imposes hard limits on evolutionary search: although monotonic functions like tanh can represent parity given enough width, neuroevolution in

sparse substrates fails to discover such solutions, while a single oscillatory unit suffices. We formalize this gap in this paper.

Can indirect encoding evolve per-node activation function assignments that match computational substrates to problem structure? This question matters because activation function selection is not hyperparameter optimization: it determines what a network *can* represent. Choosing the wrong function category makes a problem unsolvable; choosing the right one can make it trivial.

Why use evolutionary search rather than hand-designed topologies? Assigning heterogeneous activation functions per node creates a combinatorial space that grows exponentially with network size. Indirect encoding via a Compositional Pattern Producing Network (CPPN) (Stanley, 2007) makes this search tractable: the CPPN generates the substrate topology and connection weights from spatial coordinates, which we extend to also assign each node an activation function, so that compact genomes can produce large structured networks with heterogeneous functions. Evolution discovers function assignments unlikely to be chosen by hand.

We test three assignment methods (CPPN-output, weight-interpretation, uniform) across 18 activation functions spanning 7 categories, and compare evolutionary search with gradient-based optimization, revealing that the solvability divide reverses under gradient descent. We address three research questions:

- **RQ1:** Can neuroevolution with indirect encoding discover per-node activation assignments, and does the choice of function category create hard solvability limits?
- **RQ2:** Which assignment mechanism is most effective for per-node function selection via CPPN?
- **RQ3:** Is the solvability divide universal, or specific to particular problem structures and optimizers?

## Contributions

1. **Per-function solvability characterization (RQ1):** We individually test all 18 activation functions from the

palette on Parity-4, revealing a three-tier solvability structure (100% / 6.7–80% / 0%) quantitatively predicted by local extrema count (Spearman  $\rho=0.81$ ). We separate representability from discoverability: monotonic networks can represent parity with width  $\Theta(n)$ , yet neuroevolution does not find it in sparse substrates, while a single oscillatory unit suffices (Propositions 1, 2).

2. **Assignment method and palette robustness** (RQ2): CPPN-controlled per-node assignment significantly outperforms heuristic and uniform methods, larger palettes slow convergence without reducing success, and the divide persists across population sizes 50–2,000.
3. **Boundary conditions** (RQ3): Recurrence collapses the divide (monotonic 0%→80–100% on Parity-4), gradient descent inverts it (tanh maintains 100% through Parity-8), and the divide is absent on non-parity tasks (all 18 functions solve sine regression at 100%). The divide is specific to evolutionary search in indirect encoding, not universal.

## Background

### Indirect Encoding and HyperNEAT

NEAT (Stanley and Miikkulainen, 2002; Papavasileiou et al., 2021) is a direct encoding that evolves neural network topology and weights simultaneously through speciation and incremental complexification. HyperNEAT (Stanley et al., 2009) extends NEAT with indirect encoding: NEAT evolves a compact CPPN (Stanley, 2007) that generates connection weights from spatial coordinates  $w_{ij} = \text{CPPN}(x_i, y_i, x_j, y_j)$ , exploiting geometric regularities (Gauci et al., 2008) to produce large structured networks from small genomes. ES-HyperNEAT (Risi and Stanley, 2012) adds evolvable substrates via variance-based neuron placement. EMR-HyperNEAT (Claret et al., 2026b) replaces ES-HyperNEAT’s sequential quadtree with a dense multi-resolution grid evaluated eagerly as tensor operations, for GPU acceleration.

Standard HyperNEAT applies uniform activation (tanh) to all substrate nodes. We extend this to allow per-node activation selection from a predefined palette of 18 functions.

### Activation Function Diversity

Activation function handling in the NEAT lineage has progressed through three levels. *Level 1 (fixed)*: Original NEAT (Stanley and Miikkulainen, 2002) applies a steepened sigmoid to every node; no per-node diversity exists. *Level 2 (random-at-creation, CPPN only)*: HyperNEAT (Stanley et al., 2009) assigns each CPPN node a random activation function at creation, but the decoded substrate uses uniform activation. *Level 3 (mutable per-node)*: HA-NEAT (Hagg et al., 2017) and HAFD-NEAT (Papavasileiou et al., 2022) evolve per-node activations via direct encoding. We bring

Level 3 to indirect encoding: a CPPN output assigns activation functions to substrate nodes from spatial coordinates, which produces spatially structured functional regions without per-gene mutation.

We are not aware of prior work that evolves per-node substrate activation via indirect encoding. We also give a formal account of both sides of the oscillatory/monotonic divide and characterize all 18 functions individually.

## Related Work

**Heterogeneous activations.** All prior per-node activation work operates in direct encoding (Hagg et al., 2017; Papavasileiou et al., 2022). The indirect-encoding mechanism presented here produces spatially structured functional regions (described in the Methods). Our results extend Hagg et al.’s finding that mixed activations improve parsimony: we show function *category* creates a categorical solvability divide that goes beyond a size advantage. Prior gradient-based approaches (PANGAEA (Bingham and Miikkulainen, 2022), Ramachandran et al. (Ramachandran et al., 2017)) search for a single best function. We evolve per-node heterogeneous assignments with formal solvability analysis.

**Periodic activations.** SIREN (Sitzmann et al., 2020) shows that periodic sinusoidal activations can enable implicit representations inaccessible to ReLU. Ziyin et al. (Ziyin et al., 2020) formally show standard networks fail to learn periodic functions. Rahaman et al. (Rahaman et al., 2019) identify spectral bias: deep ReLU networks learn low-frequency components first. The Growing Cosine Unit (Noel et al., 2021) shows that a single oscillatory neuron solves XOR by creating multiple decision boundaries, in line with our Proposition 1 (presented in the Formal Analysis). Sopena et al. (Sopena et al., 1999) show that periodic perceptrons can compute arbitrary Boolean functions. Our Propositions 1 and 2 formalize both directions of the divide.

**Architecture search.** NAS (Zoph and Le, 2016; Real et al., 2019) and CoDeepNEAT (Miikkulainen et al., 2024) search architectures including activations at the layer level. Nader and Azar (Nader and Azar, 2021) survey evolved activation functions. We instead assign activation functions per node, via indirect encoding.

## Methods

### System Pipeline

Figure 1 shows the evolutionary pipeline. Each individual encodes a CPPN genome that generates a substrate by querying connection weights from spatial coordinates. The CPPN also assigns a per-node activation function from the palette: each substrate node receives its own activation function. The substrate is evaluated on the target problem, and standard NEAT selection produces the next generation.

### Three Per-Node Assignment Methods

**CPPN-output (Method A).** The CPPN produces an extra output channel beyond connection weights. The continuous value  $x$  is squashed by  $\tanh$  and binned into one of the  $k$  palette indices:  $\text{idx} = \lfloor (\tanh(x)+1) k/2 \rfloor$  (clipped to  $[0, k-1]$ ). Spatial patterns in the CPPN naturally create structured functional regions.

**Weight-interpretation (Method B).** Activation function indices are derived from incoming weight statistics without additional CPPN outputs, mapping a continuous statistic to a palette index (not a hard threshold). Three variants:

- *sign-based*: positive weights  $\rightarrow \tanh$ , negative  $\rightarrow \text{relu}$ , mixed  $\rightarrow \text{sigmoid}$ ;
- *magnitude-based*: larger mean  $|w| \rightarrow \text{bounded}$ ;
- *variance-based*: larger weight variance  $\rightarrow \text{nonlinear}$ , e.g.  $\tanh(5 \text{Var}[w])$ .

**Uniform (Method C).** All substrate nodes receive the same activation function. This serves as the baseline.

### Function Palette: 18 Activation Functions

Table 1 lists all 18 functions organized into 7 categories. The palette spans oscillatory functions (which cross decision thresholds multiple times), monotonic functions (which cross at most once), and intermediate categories.

Table 1: 18 activation functions and their individual Parity-4 results (Per-Function Ablation;  $N=30$  seeds; 300 gen max). The solvability boundary follows oscillatory structure: functions with local extrema solve parity; 9 strictly monotonic functions never solve.

| Idx                                              | Name      | Formula                   | Category        | Solve%     | Avg Gen |
|--------------------------------------------------|-----------|---------------------------|-----------------|------------|---------|
| <i>Multi-crossing</i> (100% solve, 2.9–69.2 gen) |           |                           |                 |            |         |
| 4                                                | sin       | $\sin(x)$                 | Oscillatory     | <b>100</b> | 2.9     |
| 13                                               | osc.adapt | $\sin(x)(1-0.2 x )$       | Oscillatory     | <b>100</b> | 3.7     |
| 11                                               | burst     | $\tanh(x)+0.5\sin(3x)$    | Oscillatory     | <b>100</b> | 10.7    |
| 16                                               | band_pass | $e^{- x-1 } - e^{- x+1 }$ | Adaptive        | <b>100</b> | 69.2    |
| <i>Intermediate</i> (6.7–80% solve, 55–246 gen)  |           |                           |                 |            |         |
| 15                                               | receptive | $e^{-x^2} \cos(2x)$       | Quasi-Osc.      | 80         | 100.1   |
| 12                                               | resonator | $\sin(x) e^{- x /3}$      | Oscillatory     | 66.7       | 55.9    |
| 5                                                | gauss     | $e^{-x^2}$                | Radial          | 33.3       | 139.3   |
| 8                                                | rs.adapt  | $\tanh(x)(1-0.3 x )$      | Adaptive        | 23.3       | 202.4   |
| 10                                               | lts.low   | $\sigma(2x-0.5)$          | Mono./Bounded   | 6.7        | 246.0   |
| <i>Monotonic</i> (0% solve, 9 functions)         |           |                           |                 |            |         |
| 0                                                | tanh      | $\tanh(x)$                | Mono./Bounded   | 0          | —       |
| 1                                                | sigmoid   | $\sigma(x)$               | Mono./Bounded   | 0          | —       |
| 2                                                | relu      | $\max(0, x)$              | Mono./Unbounded | 0          | —       |
| 3                                                | identity  | $x$                       | Linear          | 0          | —       |
| 6                                                | lelu      | $\max(0.01x, x)$          | Mono./Unbounded | 0          | —       |
| 7                                                | softplus  | $\ln(1 + e^x)$            | Mono./Unbounded | 0          | —       |
| 9                                                | fs.fast   | $2\text{relu}(x)$         | Mono./Unbounded | 0          | —       |
| 14                                               | gain_mod  | $x/(1+ x )$               | Adaptive        | 0          | —       |
| 17                                               | integrate | $\tanh(x)(1+0.2e^{- x })$ | Mono./Bounded   | 0          | —       |

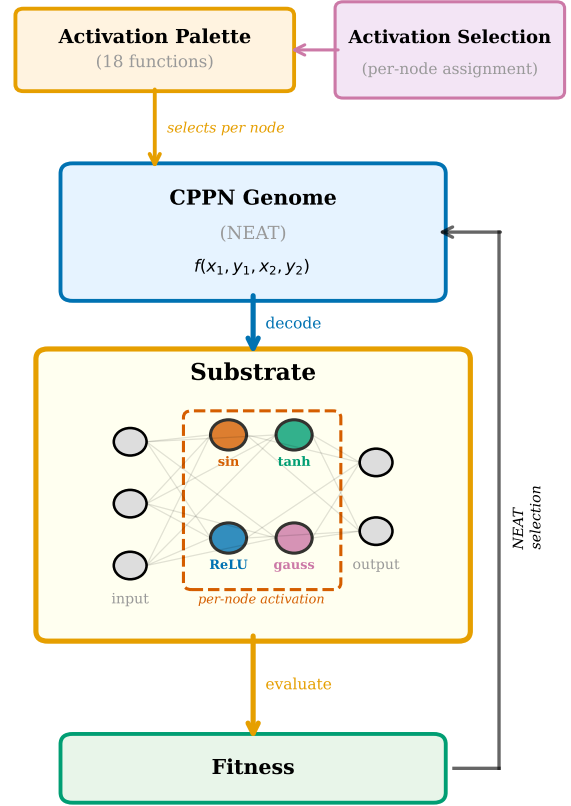


Figure 1: System pipeline. The activation palette, through the per-node selection mechanism, assigns each node its own activation function; the CPPN genome (NEAT) is decoded into a substrate carrying these per-node activations, the substrate is evaluated for fitness, and NEAT selection closes the evolutionary loop. Three assignment methods determine how CPPN output maps to function selection.

### Experimental Setup

Experiments span logic problems (XOR, Parity-2 through Parity-8), classification (Two Spirals, concentric circles, step function), continuous regression (sine approximation), and multi-step reasoning (Tower of Hanoi). All use feedforward substrate topology unless noted, 30 independent replications (random seeds 42–71), and a 0.95 fitness threshold for “solved” (0.90 for concentric circles).

**Reproducibility.** Each individual is a NEAT-evolved CPPN that emits connection weights and, in CPPN-output mode, one extra channel decoded to a per-node activation index. Candidate substrate nodes occupy a multi-resolution grid, where level  $i$  contributes  $4^{i+1}$  cells (e.g., 84 positions at depth 2, 21,844 at depth 6); CPPN-variance filtering (threshold 0.03) retains the informative ones. NEAT uses connection/node add–delete mutation (probabilities 0.5/0.5/0.2/0.2), compatibility threshold 3.0, genome elitism 15, survival threshold 0.2, and CPPN node activations drawn

from  $\{\tanh, \sin, \text{gauss}\}$ . Unless noted, the population is 150 and each run is capped at 300 generations. The Population Size Invariance study instead sweeps the population over 50, 150, 500, 1,000, and 2,000; it and the Parity-scaling study use a 100-generation cap. Recurrent runs add hidden-to-hidden, backward, lateral, and self connections and iterate the substrate deterministically for  $2^d+1$  steps at depth  $d$ . Complete configurations and result data are in the repository (Data/Code link above).

All group comparisons use Mann–Whitney  $U$  tests (two-group) or Fisher’s exact tests (proportions); correlations use Spearman’s rank coefficient. Unless noted otherwise, reported differences are statistically significant at  $p < 0.01$ .

## Results

### Per-Function Solvability

Parity- $N$  problems (output 1 if an odd number of  $N$  binary inputs are active) require alternating decision boundaries. Testing activation functions on Parity-4 reveals a three-tier solvability structure (Table 1).

On XOR (simplest parity),  $\sin$  converges  $16\times$  faster than a monotonic-only baseline palette ( $\tanh$ , sigmoid,  $\text{relu}$ , identity): 1.1 vs 17.5 generations (Mann–Whitney  $U=18$ ,  $p=1.67\times 10^{-11}$ , rank-biserial  $r=0.96$ ). Gaussian and sinusoidal activations are both smooth and bounded, yet  $\sin$  achieves 100% on parity while Gaussian achieves only 33%. Both cross zero, but only  $\sin$  does so *periodically*. The decisive property is periodicity, not smoothness.

In evolved substrates with sparse hidden layers, the few hidden nodes produced by CPPN variance analysis fall short of the width- $\Theta(n)$  monotonic construction (Proposition 2), which evolution does not discover. This divide is not universal (see the Discussion).

### Formal Analysis

The oscillatory/monotonic divide we observe is a barrier of *search and sparsity*, not of representation. We make both sides precise and locate the gap.

**Proposition 1 (Oscillatory sufficiency)** *A single hidden unit with cosine activation computes  $n$ -bit parity  $P_n$  for any  $n$ . Let  $h(\mathbf{x}) = \cos(\pi \sum_{i=1}^n x_i)$ . For  $\mathbf{x} \in \{0, 1\}^n$ ,  $\sum_i x_i = k \in \{0, \dots, n\}$ , so  $h(\mathbf{x}) = \cos(\pi k) = (-1)^k$ :  $+1$  when  $k$  is even and  $-1$  when odd. The linear readout  $y = \frac{1}{2}(1 - h(\mathbf{x}))$  outputs  $P_n$  with a single hidden unit, independent of  $n$ .*

**Proposition 2 (Monotonic feasibility)** *A single monotonic unit (perceptron) cannot compute  $P_n$  for  $n \geq 2$  (Minsky and Papert, 2017), but a single hidden layer of monotonic units can, with width  $\Theta(n)$ . The “thermometer” construction uses  $n$  threshold units  $h_i = 1[\sum_j x_j \geq i]$  and a sign-alternating readout  $y = \sum_{i=1}^n (-1)^{i+1} h_i$ ; if  $s = \sum_j x_j$  then  $y = \sum_{i=1}^s (-1)^{i+1} = P_n(\mathbf{x})$ . Parity is thus representable by monotonic networks of modest width.*

These bounds concern representation. The phenomenon our experiments expose lies between representability and *discoverability under evolutionary search in sparse substrates*. Variance-based substrate discovery yields few hidden nodes, so the width- $\Theta(n)$  monotonic construction of Proposition 2, which also requires coordinating the alternating readout weights, is not found by neuroevolution, while a single oscillatory unit (Proposition 1) needs neither width nor coordination and is found within a few generations. Gradient descent with a fixed wide layer reaches the monotonic solution reliably ( $\tanh$  solves Parity-8 with  $h=32 < 2^7$  units; Table 10), confirming the barrier is search and sparsity, not representation.

### Per-Function Ablation

We test each of the 18 palette functions individually on Parity-4 using uniform activation mode (population 150, 300 generation maximum). Table 1 shows both function definitions and results.

The solvability boundary follows oscillatory structure, quantified by local extrema count (Spearman  $\rho=0.81$ ,  $p=4.4\times 10^{-5}$ ), not category label. Four functions achieve 100%:  $\sin$  (2.9 gens),  $\text{osc\_adapt}$  (3.7),  $\text{burst}$  (10.7), and  $\text{band\_pass}$  (69.2). Five functions form an intermediate tier.  $\text{Rs\_adapt}$  (23.3%) has hidden oscillatory behavior:  $\tanh(x)(1-0.3|x|)$  reverses sign at  $|x| \approx 3.3$ .  $\text{Lts\_low}$  (6.7%), a shifted sigmoid, is genuinely monotonic, solving only by threshold proximity rather than true parity representation, consistent with Proposition 2. All 9 remaining strictly monotonic functions achieve 0%. However, monotonic functions are not helpless: their mean plateau fitness is  $0.851 \pm 0.048$  ( $\sim 85\%$  of patterns correct on Parity-4,  $N=270$  across 9 functions). Evolution gets close but cannot cross the 0.95 threshold, which requires resolving the alternating parity structure that monotonic units capture only with width (Proposition 2). The divide is depth-invariant:  $\sin$  achieves 100% and  $\tanh$  0% at depths 2, 4, and 6. A  $260\times$  increase in available substrate capacity does not rescue monotonic functions, indicating an evolvability barrier rather than a capacity shortage, consistent with Proposition 2, where monotonic parity needs width that variance-filtered substrates and evolutionary search do not supply. Measuring the evolved substrates directly confirms this: the  $\sin$  networks that solve parity use a median of 15 hidden nodes and  $\tanh$ ’s best-of-failed networks a comparable 10 hidden nodes, both far exceeding the few units the monotonic construction needs, yet  $\tanh$  never solves. The connection counts agree:  $\tanh$ ’s failed substrates carry a median of 44.5 connections (range 2–96), again more than the sparse monotonic construction requires, even though they stay below the 100-connection budget that most  $\sin$  solutions reach. Figure 2 quantifies this: a gradient from undamped oscillations (100%) to strictly monotonic (0%).

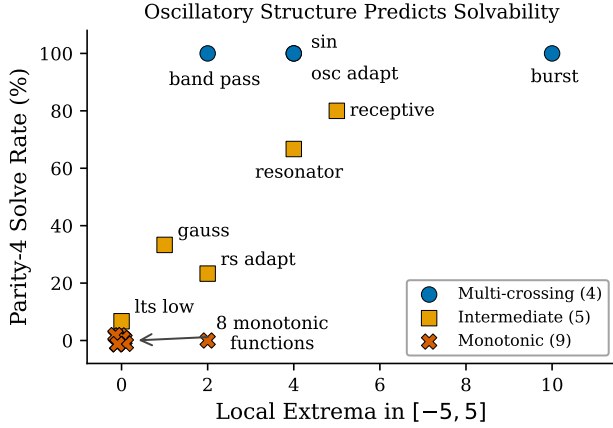


Figure 2: Oscillatory structure predicts solvability. Local extrema count in  $[-5, 5]$  vs Parity-4 solve rate ( $N=30$ ) for all 18 activation functions. Functions with more local extrema achieve higher solve rates. The two outliers at 2 extrema reflect extrema amplitude: `band_pass` has deep sign-reversing extrema (100%), while `integrate` has shallow perturbations of a monotonic trend (0%).

**Per-function testing on XOR.** Repeating the per-function test on XOR (300 gen max) preserves the three-tier structure with shifted thresholds: all 7 oscillatory functions solve 100%, monotonic functions only partially (`tanh` 70%, `relu` 40%, `sigmoid` 26.7%, `identity` 0%). XOR is equivalent to Parity-2, and the higher `tanh` rate here (70% vs 50% in Table 2) reflects the longer generation limit (300 vs 100) and XOR being the easiest parity instance; the added search budget mitigates but does not eliminate the monotonic barrier. Per-function solve rates correlate strongly across the two tasks (Spearman  $\rho=0.84$ ), so the divide is not task-specific.

### Parity Scaling

Using `sin` activation in uniform mode, we evolve independent populations on Parity-2 through Parity-8. Table 2 shows the results.

Table 2: Parity scaling: `sin` vs `tanh` ( $N=30$  seeds each; population 150; 100 gen max). `Sin` achieves 100% across all sizes; `tanh` degrades monotonically. Percentages are solve rates (replications reaching the threshold).

| Problem  | Samples | Sin% | Sin Gen | Tanh% | Tanh Fit. |
|----------|---------|------|---------|-------|-----------|
| Parity-2 | 4       | 100  | 1.0     | 50.0  | 0.930     |
| Parity-3 | 8       | 100  | 2.2     | 3.3   | 0.890     |
| Parity-4 | 16      | 100  | 2.9     | 0     | 0.863     |
| Parity-5 | 32      | 100  | 2.9     | 0     | 0.826     |
| Parity-6 | 64      | 100  | 3.4     | 0     | 0.825     |
| Parity-7 | 128     | 100  | 3.7     | 0     | 0.821     |
| Parity-8 | 256     | 100  | 3.6     | 0     | 0.791     |

*Sin:* 100% across all sizes, from 4 to 256 samples, within 1–4 generations on average, consistent with Proposition 1. *Tanh:* monotonic degradation. Even on Parity-2,

`tanh` achieves only 50%; from Parity-3 onward, categorically 0%. Mean best fitness degrades from 0.930 (Parity-2) to 0.791 (Parity-8), consistent with the width-and-coordination barrier of Proposition 2. Convergence speed for `sin` is flat (1.0–3.7 gens across  $64 \times$  sample range), suggesting the difficulty scales sublinearly with problem size.

### Population Size Invariance

Larger populations explore more of the weight space, potentially discovering the width- $\Theta(n)$  monotonic construction (Proposition 2). We re-evaluate `sin` vs `tanh` on Parity-4 with the population genuinely varied from 50 to 2,000 (Table 3).

Table 3: Population reshapes the dynamics but not the oscillatory divide (Parity-4,  $N=30$  per cell, 100 gen max). `Sin` solves 100% at every size; `tanh` 0% at every size. Percentages are solve rates.

| Pop Size | Sin% | Sin Gen | Tanh% | Tanh Fit. |
|----------|------|---------|-------|-----------|
| 50       | 100  | 5.0     | 0     | 0.808     |
| 150      | 100  | 3.0     | 0     | 0.863     |
| 500      | 100  | 2.0     | 0     | 0.894     |
| 1,000    | 100  | 1.0     | 0     | 0.900     |
| 2,000    | 100  | 1.0     | 0     | 0.910     |

Across a  $40 \times$  range, the solve-rate divide is invariant: `sin` 100% and `tanh` 0% at every population. Population is not inert, however. It *accelerates* `sin`: median convergence falls from 5 generations (pop 50) to 1 (pop  $\geq 1,000$ ) as larger pools sample a working oscillatory unit sooner. It also lifts `tanh`’s best-found fitness monotonically from 0.808 to 0.910, yet `tanh` never crosses the 0.95 threshold at any size. Added search budget brings monotonic networks closer to parity but supplies neither the width nor the weight coordination that Proposition 2 requires.

### Assignment Method Comparison

Having established the solvability divide on parity (RQ1), we turn to RQ2: which assignment mechanism best places per-node functions? We use a spatial problem rather than parity, since assignment matters most where function choice interacts with geometry. Two Spirals (194 interleaved points, binary classification) provides a non-trivial testbed for comparing the three assignment methods. Table 4 shows results from 9 configurations tested at depth 3.

Weight-interpretation provides zero benefit over no function selection: all three variants collapse to the 75.2% uniform baseline. CPPN-controlled assignment yields only a marginal gain on Two Spirals (76.9% vs 75.2%), near the feedforward ceiling. The decisive evidence is on concentric circles, where the effect is categorical: CPPN-output with the 6-function palette achieves 66.7% vs 0% for both weight-interpretation and uniform methods (Fisher’s exact  $p=1.4 \times 10^{-8}$ ). Adding `sin` to the CPPN palette *hurts* Two Spirals (76.1% vs 76.9%), consistent with substrate match-

Table 4: Assignment method comparison on Two Spirals ( $N=30$ ; depth 3). CPPN-output significantly outperforms the other methods; weight-interpretation gives no gain over the uniform baseline.  $\text{cppn\_output}_k$  = CPPN-output over a  $k$ -function palette (4: monotonic only; 6: adds sin, gauss).

| Method         | Configuration   | Accuracy     | Std  |
|----------------|-----------------|--------------|------|
| CPPN-output    | cppn_output_4   | <b>76.9%</b> | 0.29 |
|                | cppn_output_6   | <b>76.9%</b> | 0.29 |
|                | cppn_with_sin   | 76.1%        | 0.42 |
| Weight-interp. | sign-based      | 75.2%        | 0.14 |
|                | magnitude-based | 75.2%        | 0.14 |
|                | variance-based  | 75.2%        | 0.14 |
| Uniform        | disabled        | 75.2%        | 0.14 |
|                | global_sin      | 75.2%        | 0.14 |
|                | global_tanh     | 75.2%        | 0.14 |

ing: oscillatory functions are mismatched to spatial classification.

### Palette Size Robustness

How sensitive is performance to palette size? Table 5 shows that all palette sizes from 1 to 13 of the 18 available functions achieve 100% success rate on Parity-4. The search space expansion argument ( $k^N$  configurations) predicts exponential degradation; success rate is unaffected, though convergence slows  $5.5\times$  from 1 to 13 functions ( $1.7\rightarrow 9.3$  generations, with standard deviation increasing from 0.7 to 6.4).

Table 5: Palette size vs convergence speed ( $N=30$  seeds per size; Parity-4; all 100% success).

| Palette Size | Functions       | Avg Gen | Std |
|--------------|-----------------|---------|-----|
| 1 (sin only) | sin             | 1.7     | 0.7 |
| 3            | sin, tanh, relu | 3.4     | 1.2 |
| 4            | +sigmoid        | 3.1     | 1.1 |
| 5            | +identity       | 3.2     | 1.2 |
| 6            | +gauss          | 2.6     | 1.1 |
| 7            | +lelu           | 3.3     | 1.0 |
| 8            | +softplus       | 3.4     | 1.2 |
| 13           | 13 functions    | 9.3     | 6.4 |

### Recurrence Collapses the Divide

Table 6 compares each function’s solve rate under feedforward versus recurrent topology on Parity-4 (threshold 0.95). Recurrence partially collapses the divide: all oscillatory and quasi-oscillatory functions remain at 100%, and *lts.low* rises from 6.7% $\rightarrow$ 100%. Among monotonic functions, the collapse is graded: *softplus* 0% $\rightarrow$ 90%, *sigmoid* 0% $\rightarrow$ 70%, *gauss* 33% $\rightarrow$ 77%, but *tanh* remains near 0% (6.7%) and *identity*, *lelu* stay at 0%. Recurrence gives bounded monotonic functions temporal iteration sufficient for some seeds, but unbounded functions (*relu* 3.3%, *fs.fast* 6.7%, *identity* 0%, *lelu* 0%) cannot benefit: their unbounded

self-application diverges rather than producing oscillatory dynamics.

Table 6: Feedforward vs recurrent topology on Parity-4 (both  $N=30$ , 300 gen max, pop 150; solve threshold 0.95). Recurrence partially collapses the divide: oscillatory functions remain at 100%; bounded monotonic partially solve; unbounded/linear remain near 0%. % = solve rate.

| Function  | FF%  | FF Gen | Rec%        | Rec Gen |
|-----------|------|--------|-------------|---------|
| sin       | 100  | 2.9    | 100         | 2.0     |
| osc_adapt | 100  | 3.7    | 100         | 3.0     |
| burst     | 100  | 10.7   | 100         | 5.0     |
| band_pass | 100  | 69.2   | 100         | 10.0    |
| receptive | 80   | 100.1  | <b>100</b>  | 3.0     |
| resonator | 66.7 | 55.9   | <b>100</b>  | 2.0     |
| gauss     | 33.3 | 139.3  | <b>76.7</b> | 95.0    |
| rs_adapt  | 23.3 | 202.4  | <b>30.0</b> | 165.0   |
| lts_low   | 6.7  | 246.0  | <b>100</b>  | 4.0     |
| softplus  | 0    | —      | <b>90.0</b> | 7.0     |
| sigmoid   | 0    | —      | <b>70.0</b> | 7.0     |
| integrate | 0    | —      | <b>30.0</b> | 90.0    |
| gain_mod  | 0    | —      | <b>13.3</b> | 6.5     |
| tanh      | 0    | —      | 6.7         | 98.5    |
| fs_fast   | 0    | —      | 6.7         | 210.0   |
| relu      | 0    | —      | 3.3         | 271.0   |
| identity  | 0    | —      | 0           | —       |
| lelu      | 0    | —      | 0           | —       |

**Parity-8 with recurrence.** Under full recurrence on Parity-8, all 4 oscillatory functions still solve 100%, and the monotonic collapse stays category-dependent: bounded/radial functions partially solve (32.2%; *sigmoid* 70%, *softplus* 50%, *gauss* 27%) while unbounded/linear remain at 0%.

**Which connection types matter?** Table 7 decomposes full recurrence by connection type (*tanh* on Parity-4, 0.95 threshold). Hidden-to-hidden connections alone do most of the work (20% vs 0% feedforward), while backward, lateral, and self-loops add little (6.7–13.3%).

Table 7: Connection type decomposition (*tanh* on Parity-4,  $N=30$ , 300 gen max, pop 150; solve threshold 0.95). Each row adds one connection type to hidden-to-hidden (h-to-h) as base.

| Connection Type     | Solve%      | Mean Gen | Avg Fitness |
|---------------------|-------------|----------|-------------|
| feedforward only    | 0           | —        | 0.914       |
| h-to-h only         | <b>20.0</b> | 79.5     | 0.911       |
| h-to-h + backward   | 13.3        | 191.2    | 0.917       |
| h-to-h + lateral    | 6.7         | 117.5    | 0.881       |
| h-to-h + self-loops | 6.7         | 58.5     | 0.885       |
| all recurrent       | 6.7         | 98.5     | 0.893       |

**Ceilings persist under recurrence.** Despite recurrence partially collapsing the solvability divide on parity, feedfor-

ward performance ceilings survive elsewhere. Across 240 recurrent runs on Tower of Hanoi, 0% solve. On Two Spirals, 0% across 240 runs. On concentric circles, recurrence has the *opposite* effect across all 8 tested functions (Table 8; in both full-recurrence and hidden-only modes).

Table 8: Recurrence on concentric circles ( $N=30$  each; solve threshold 0.90). Opposite to parity, recurrence *degrades* most functions; only `osc_adapt` stays at 100%. FF% = feedforward solve rate; Full Rec% = full recurrence; Hidden-only% = recurrence on hidden-to-hidden connections only.

| Function               | FF% | Full Rec%  | Hidden-only% |
|------------------------|-----|------------|--------------|
| <code>osc_adapt</code> | 100 | <b>100</b> | <b>100</b>   |
| <code>sin</code>       | 100 | 80         | 50           |
| <code>resonator</code> | 100 | 70         | 57           |
| <code>gauss</code>     | 43  | 3.3        | 3.3          |
| <code>burst</code>     | 0   | 0          | 0            |
| <code>sigmoid</code>   | 3.3 | 0          | 6.7          |
| <code>softplus</code>  | 0   | 0          | 6.7          |
| <code>tanh</code>      | 0   | 0          | 0            |

`Osc_adapt` alone is immune to recurrence-induced degradation on circles: its damped oscillation provides spatial locality that survives recurrent iteration, while pure `sin` (80%) and `resonator` (70%) degrade, and monotonic functions stay near 0%. Substrate matching thus needs *architecture-problem* alignment too: temporal iteration helps alternating boundaries (parity) but disrupts the spatial coherence that radial enclosure (circles) needs.

### Non-Parity Generalization

Does the divide generalize beyond parity? We test all 18 functions on three non-parity problems: sine regression (continuous periodic), step function (linear classification), and concentric circles (radial classification). Table 9 summarizes results for representative functions across all four problem types.

**Sine regression.** All 18 functions achieve 100%. The divide vanishes when smooth interpolation replaces discrete boundary partitioning.

**Concentric circles.** The three-tier structure replicates. Radial boundaries, like parity, require non-monotonic decision surfaces: oscillatory functions solve (`osc_adapt` 100%, `sin` 100%), while most monotonic functions fail (7 of 9 at 0%; `sigmoid` and `relu` reach 3.3%).

**Step function.** The pattern inverts. Identity converges fastest (median 4 gens). Monotonic functions outperform oscillatory ones (`sin` median 55 gens), while localized functions like `gauss` struggle (13%) on this global linear boundary. Oscillatory structure is counterproductive when the problem is linearly separable.

Table 9: Cross-problem comparison for representative functions. The three-tier solvability structure replicates on problems requiring non-monotonic decision surfaces (Parity-4, Circles) and vanishes on smooth problems (Sine, Step). All cells  $N=30$ . % = solve rate; Gen = median generations to solve.

| Function               | Parity-4 |      | Sine Regr. |     | Step Func. |     | Circles |     |
|------------------------|----------|------|------------|-----|------------|-----|---------|-----|
|                        | %        | Gen  | %          | Gen | %          | Gen | %       | Gen |
| <code>sin</code>       | 100      | 2.9  | 100        | 1   | 97         | 55  | 100     | 23  |
| <code>osc_adapt</code> | 100      | 3.7  | 100        | 1   | 100        | 49  | 100     | 2   |
| <code>burst</code>     | 100      | 10.7 | 100        | 6   | 100        | 24  | 0       | —   |
| <code>resonator</code> | 67       | 56   | 100        | 1   | 100        | 50  | 100     | 8   |
| <code>gauss</code>     | 33       | 139  | 100        | 2   | 13         | 152 | 43      | 155 |
| <code>rs_adapt</code>  | 23       | 202  | 100        | 1   | 97         | 52  | 47      | 149 |
| <code>tanh</code>      | 0        | —    | 100        | 1   | 100        | 18  | 0       | —   |
| <code>sigmoid</code>   | 0        | —    | 100        | 10  | 100        | 38  | 3.3     | 121 |
| <code>identity</code>  | 0        | —    | 100        | 2   | 100        | 4   | 0       | —   |
| <code>relu</code>      | 0        | —    | 100        | 6   | 83         | 14  | 3.3     | 271 |

The divide emerges wherever problems require non-monotonic decision surfaces and disappears on smooth problems: match function complexity to *decision boundary complexity*.

## Discussion

### The Divide Is Optimizer-Specific

Is the oscillatory/monotonic divide universal or specific to evolutionary search? We trained standard MLPs (single hidden layer with  $h$  hidden units, Adam optimizer,  $lr=0.01$ , 5,000 epochs max) with `sin` vs `tanh` activation on parity tasks at multiple values of  $h$  (Table 10).

On Parity-4 and Parity-5, both activations achieve 100% at all tested  $h$ , with `tanh` converging faster.

From Parity-6 onward, the divide progressively reverses. `Sin` degrades from 100% (Parity-5) to 47–67% (Parity-6), 10–37% (Parity-7), and 0% (Parity-8, including  $h=64$ ), while `tanh` stays at 100%. Counterintuitively, *more* hidden units hurt `sin` (Parity-6: 66.7% at  $h=16$  vs 46.7% at  $h=32$ ): Xavier initialization (Glorot and Bengio, 2010) concentrates pre-activations near zero where  $\sin(x) \approx x$ , and small gradient steps never leave that quasi-linear regime (Sitzmann et al., 2020; Ziyin et al., 2020). Indirect encoding sidesteps this: CPPN-generated and NEAT-mutated weights span the full range, so `sin` starts in its oscillatory regime, while gradient descent coordinates the  $\Theta(n)$  monotonic units of Proposition 2 that evolutionary search cannot.

### Practical Implications

Our results yield concrete guidelines for neuroevolution with indirect encoding:

**Function-problem matching.** If a problem requires multi-crossing decision boundaries (parity, XOR chains,

Table 10: Gradient-trained MLP baseline (Adam,  $lr=0.01$ , 5,000 epochs max,  $N=30$ ). The divide progressively reverses: tanh maintains 100% through Parity-8; sin degrades to 0%. Increasing  $h$  does not help sin; it makes it worse on Parity-6 (66.7%→46.7%).

| Problem  | Act. | $h$ | Solve% | Avg Epoch |
|----------|------|-----|--------|-----------|
| Parity-4 | sin  | 16  | 100    | 257       |
| Parity-4 | tanh | 16  | 100    | 182       |
| Parity-5 | sin  | 16  | 100    | 477       |
| Parity-5 | sin  | 32  | 100    | 543       |
| Parity-5 | tanh | 16  | 100    | 245       |
| Parity-6 | sin  | 16  | 66.7   | 2,653     |
| Parity-6 | sin  | 32  | 46.7   | 2,986     |
| Parity-6 | tanh | 16  | 100    | 352       |
| Parity-6 | tanh | 32  | 100    | 370       |
| Parity-7 | sin  | 16  | 36.7   | 3,805     |
| Parity-7 | sin  | 32  | 10.0   | 3,217     |
| Parity-7 | tanh | 16  | 100    | 563       |
| Parity-7 | tanh | 32  | 100    | 572       |
| Parity-8 | sin  | 32  | 0      | —         |
| Parity-8 | sin  | 64  | 0      | —         |
| Parity-8 | tanh | 32  | 100    | 1,078     |

alternating patterns), include oscillatory functions, since monotonic-only palettes will fail under evolutionary search. For smooth regression or linear classification, function category is irrelevant: all 18 functions solve sine regression at 100%. Among the 18 functions, osc.adapt alone maintains 100% across all tested conditions (parity, circles, feedforward, recurrent), making it a safe default.

**Assignment method.** CPPN-output assignment outperforms weight-interpretation and uniform methods. Add an extra CPPN output for function selection rather than deriving it from weights.

**Palette sizing.** Larger palettes slow convergence ( $5.5\times$  from 1 to 13 functions on Parity-4) but do not reduce success rate. When problem structure is unknown, use the full palette. Slower convergence is acceptable insurance against choosing the wrong category.

## Limitations

**Task scope.** Our evidence spans low-dimensional problems ( $\leq 8$  inputs). High-dimensional tasks (vision, robotics) may blur the divide. Prior work on ES-HyperNEAT achieved 29% on MNIST (Claret et al., 2024); even partitioning the input into spatially specialized experts reaches only 49% (Claret et al., 2026a), far below the 98%+ of gradient methods.

**Representational scope.** Our feedforward results reflect evolved substrates with few hidden nodes; monotonic functions can represent parity given sufficient width (Proposi-

tion 2), but neuroevolution did not discover that construction in our experiments.

**Architectural constraints.** The Hanoi plateau ( $\sim 78\%$ ) and Two Spirals ceiling ( $\sim 77\%$ ) persist under both feedforward and recurrent architectures, indicating deeper problem-architecture mismatches that topology alone cannot resolve.

**Biological parallels.** The evolved functional heterogeneity mirrors a cortical principle: distinct neuron types coexist because different computational demands require different response properties, not because one type is universally better. Uniform activation is to a network what a single neuron type would be to cortex: sufficient for simple tasks, inadequate when the problem needs qualitatively different primitives. Whether evolution organizes these per-node assignments into spatially coherent functional regions is left to future work.

## Conclusion

We addressed three research questions about per-node activation function evolution in indirect encoding.

**RQ1: Categorical solvability limits.** Activation selection is substrate matching: the per-node assignment is evolved, not a tuned hyperparameter, and function category is a categorical determinant of solvability, with oscillatory functions solving parity at 100% where monotonic functions reach 0%. Both sides are formal (Propositions 1, 2): monotonic networks need width  $\Theta(n)$  that neuroevolution did not discover in our sparse-substrate experiments, while one oscillatory unit suffices. The 18-function divide is predicted by oscillatory structure ( $\rho=0.81$ ) and holds across population sizes 50–2,000.

**RQ2: Assignment mechanism.** CPPN-controlled per-node assignment significantly outperforms weight-interpretation and uniform methods.

**RQ3: Boundary conditions.** The divide is *problem-specific* (absent on sine regression, present on concentric circles), *architecture-dependent* (recurrence collapses it on parity but *widens* it on circles), and *optimizer-specific* (gradient descent inverts it: tanh solves through Parity-8 while sin degrades to 0%).

Per-node activation selection in indirect encoding lets evolution search a design space impractical by hand, shaping substrates through function category, problem geometry, and optimizer.

## Acknowledgments

Generative AI tools assisted with visualizations and draft revision.

## References

- Bingham, G. and Miikkulainen, R. (2022). Discovering parametric activation functions. *Neural Networks*, 148:48–65.
- Claret, R., Gygax, A., O’Neill, M., Cotofrei, P., and Felber, P. (2026a). Breaking the central bias: Spatially partitioned experts for coordinate-based neuroevolution. In *International conference on pattern recognition*. Springer.
- Claret, R., O’Neill, M., Cotofrei, P., and Stoffel, K. (2024). Investigating hyperparameter optimization and transferability for ES-HyperNEAT: A TPE approach. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO ’24 Companion)*, pages 1879–1887. ACM.
- Claret, R., O’Neill, M., Cotofrei, P., and Stoffel, K. (2026b). Tensor-accelerated eager multi-resolution grids for evolving large-scale substrates. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO Companion ’26)*, San Jose, Costa Rica. ACM.
- Gauci, J., Stanley, K. O., et al. (2008). A case study on the critical role of geometric regularity in machine learning. In *AAAI*, pages 628–633.
- Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings.
- Hagg, A., Mensing, M., and Asteroth, A. (2017). Evolving parsimonious networks by mixing activation functions. In *Proceedings of the genetic and evolutionary computation conference*, pages 425–432.
- Izhikevich, E. M. (2003). Simple model of spiking neurons. *IEEE Transactions on neural networks*, 14(6):1569–1572.
- LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *nature*, 521(7553):436–444.
- Markram, H., Toledo-Rodriguez, M., Wang, Y., Gupta, A., Silberberg, G., and Wu, C. (2004). Interneurons of the neocortical inhibitory system. *Nature reviews neuroscience*, 5(10):793–807.
- Miikkulainen, R., Liang, J., Meyerson, E., Rawal, A., Fink, D., Francon, O., Raju, B., Shahrzad, H., Navruzyan, A., Duffy, N., et al. (2024). Evolving deep neural networks. In *Artificial intelligence in the age of neural networks and brain computing*, pages 269–287. Elsevier.
- Minsky, M. and Papert, S. A. (2017). *Perceptrons: An Introduction to Computational Geometry*. The MIT Press.
- Nader, A. and Azar, D. (2021). Evolution of activation functions: an empirical investigation. *ACM Transactions on Evolutionary Learning and Optimization*, 1(2):1–36.
- Noel, M. M., Trivedi, A., Dutta, P., et al. (2021). Growing cosine unit: A novel oscillatory activation function that can speedup training and reduce parameters in convolutional neural networks. *arXiv preprint arXiv:2108.12943*.
- Papavasileiou, E., Cornelis, J., and Jansen, B. (2021). A systematic literature review of the successors of “neuroevolution of augmenting topologies”. *Evolutionary computation*, 29(1):1–73.
- Papavasileiou, E., Cornelis, J. P., and Jansen, B. (2022). Towards optimizing neural networks’ connectivity and architecture simultaneously with feature selection. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pages 667–670.
- Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F., Bengio, Y., and Courville, A. (2019). On the spectral bias of neural networks. In *International conference on machine learning*, pages 5301–5310. PMLR.
- Ramachandran, P., Zoph, B., and Le, Q. V. (2017). Searching for activation functions. *arXiv preprint arXiv:1710.05941*.
- Real, E., Aggarwal, A., Huang, Y., and Le, Q. V. (2019). Regularized evolution for image classifier architecture search. In *Proceedings of the aaai conference on artificial intelligence*, volume 33, pages 4780–4789.
- Risi, S. and Stanley, K. O. (2012). An enhanced hypercube-based encoding for evolving the placement, density, and connectivity of neurons. *Artificial life*, 18(4):331–363.
- Sitzmann, V., Martel, J., Bergman, A., Lindell, D., and Wetzstein, G. (2020). Implicit neural representations with periodic activation functions. *Advances in neural information processing systems*, 33:7462–7473.
- Sopena, J. M., Romero, E., and Alquezar, R. (1999). Neural networks with periodic and monotonic activation functions: a comparative study in classification problems. In *9th International Conference on Artificial Neural Networks: ICANN’99*, pages 323–328. IET.
- Stanley, K. O. (2007). Compositional pattern producing networks: A novel abstraction of development. *Genetic programming and evolvable machines*, 8(2):131–162.
- Stanley, K. O., D’Ambrosio, D. B., and Gauci, J. (2009). A hypercube-based encoding for evolving large-scale neural networks. *Artificial life*, 15(2):185–212.
- Stanley, K. O. and Miikkulainen, R. (2002). Evolving neural networks through augmenting topologies. *Evolutionary computation*, 10(2):99–127.
- Ziyin, L., Hartwig, T., and Ueda, M. (2020). Neural networks fail to learn periodic functions and how to fix it. *Advances in Neural Information Processing Systems*, 33:1583–1594.
- Zoph, B. and Le, Q. V. (2016). Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578*.